claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_2a64bc88-2a8\agent-a10fe96b7129fcbe9.jsonl`
- SHA-256: `3f8e70dca74308d99844128e3e787cd7bf1f1346f57f3bd52fdb77d99e1f7e73`
- Source modified: `2026-06-10T05:43:18+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_2a64bc88-2a8`
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`

Transcript

1. user (2026-06-10T05:41:08.948Z)

CONTEXT ? two sibling repos on a Windows machine:
- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? local AI badminton(->racket-sports) highlight indexer + rally detector. FastAPI + SQLite + ffmpeg + GPU CV via WSL; Gemini = paid cloud AI. docs/ tree is rich. Current branch docs/next-steps-multisport (master = main branch).
- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? golden-label acquisition/curation/ingestion CLI (system-of-record candidate for training corpus).
Project state (June 2026): scaffolding complete; owned-model roadmap agreed (docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md) but NO platform/owned-model implementation started; next committed platform slice = async job model. Licensing guardrail: MIT/Apache-2.0/BSD only for code+data+weights; paid LLMs are inference-only, never training-data sources.
ENVIRONMENT RULES: this dev environment has NO GPU/torch/cv2 ? do NOT attempt to run the video pipeline or import torch/cv2 modules. Test suites are offline/fully-mocked and SAFE to run. You are READ-ONLY: never edit/create/delete repo files; running tests and git read commands is allowed.
QUALITY BAR: cite file paths (and line numbers where possible) for every finding. Report what IS in the code/docs, not what you assume. Be skeptical and concrete. Your final structured output is consumed programmatically by an orchestrator ? return raw data, not prose for a human.
ROLE: adversarial verifier. An auditor ("testing") claimed the finding below. Try to REFUTE it against the actual files. Default to isReal=false if the evidence does not hold up, the behavior is documented/deliberate (e.g. listed in docs/CODE_MAP.md gotchas or DEFERRED_HARDENING_PLAN.md as a known deferral ? unless the auditor adds genuinely NEW consequence), or the severity is materially overstated.
FINDING: {"title":"wasb_hybrid.py ? the production ('live shuttle-motion substrate') segmenter ? has zero tests while its two siblings are tested; copy-paste drift would not be caught","severity":"high","area":"segmenters","file":"C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/segmenters/wasb_hybrid.py","evidence":"tests/ has test_tracknet_hybrid.py (8 tests) and test_yolo_hybrid.py (7 tests) but no test_wasb_hybrid.py; no test imports wasb_hybrid. Its own docstring (L3-7) calls it 'the live shuttle-motion substrate'. `diff wasb_hybrid.py tracknet_hybrid.py` = 191 diff lines; Phase-4 shot_count handling is structurally divergent (wasb L205-208 try/int-except vs tracknet's explicit None branch at L226-237 ? currently behaviorally equivalent, pure drift surface). There is no shared/parametrized contract test across the three hybrids, so divergence in one is invisible.","why_it_matters":"The asked-about wasb/tracknet copy-paste drift is real and tests would NOT catch it: any behavioral change to the production segmenter (candidate persistence, linking, provider handoff) ships with zero test pressure, while the non-default tracknet path is well covered ? coverage is inverted relative to production risk."}
Steps: open the cited file(s) and any counter-evidence files; verify quotes/line claims; judge severity honestly. Cheap greps allowed; do NOT run the pipeline. Return isReal, confidence, and a 1-3 sentence verdict (with corrected detail if partially right).

2. user (2026-06-10T05:41:17.386Z)

```
No files found
```

3. user (2026-06-10T05:41:18.336Z)

```
No files found
```

4. user (2026-06-10T05:41:19.520Z)

```
Found 11 files
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\BACKLOG.md
C:\Users\avidu\Projects\badminton-highlight-indexer\README.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\COMMERCIALIZATION.md
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\harvest.py
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\COMMERCIAL_READINESS_REVIEW.md
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\wasb_hybrid.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\base.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\__init__.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\gemini_refine.py
```

5. user (2026-06-10T05:41:27.828Z)

```
1       """WASB-based hybrid play segmenter.
2
3       Identical three-phase shape to TrackNetHybridSegmenter, but Phase 2 uses **WASB**
4       (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via WSL2
5       (backend/detectors/wasb_runner.py) instead of TrackNetV4. WASB ships pretrained
6       badminton weights and runs today, so this is the live shuttle-motion substrate.
7       Phase 3 (AI semantic boundaries) and Phase 4 (DB) are shared with the other hybrids.
8
9       For the per-video *calibration* goal we mainly use Phase 2 (trajectory -> candidate
10      windows); the windowing thresholds (velocity_thresh / merge_gap / min_window_duration)
11      are what get tuned against golden rally labels.
12      """
13      import os
14      import time
15      import logging
16      import concurrent.futures
17      from typing import List, Dict, Any
18
19      import cv2
20
21      from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
22      from backend.utils.video import get_video_duration, extract_video_chunk
23      from backend.sports import sport_registry
24      from backend.providers import provider_registry
25      from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
26      from backend.pipeline.detectors.base import DetectorRunner  # ABC for the de-risked
detector seam
27
28      logger = logging.getLogger(__name__)
29
30
31      def _fmt_ts(seconds: float) -> str:
32          seconds = max(0, int(seconds))
33          h, rem = divmod(seconds, 3600)
34          m, s = divmod(rem, 60)
35          return f"{h:02d}:{m:02d}:{s:02d}"
36
37
38      class WasbHybridSegmenter(BasePlaySegmenter):
39          @property
40          def name(self) -> str:
```

```
41              return "wasb_hybrid"
42
43          @property
44          def description(self) -> str:
45              return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
badminton "
46                      "weights) for precise candidate rallies + AI provider for semantic
boundaries.")
47
48          @staticmethod
49          def _probe_fps_width(video_path: str) -> tuple:
50              cap = cv2.VideoCapture(video_path)
51              fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
52              width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
53              cap.release()
54              return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
55
56          def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None,
57                            max_frames: int = None) -> List[Dict[str, Any]]:
58              sport_name = self.config.get("sport", "badminton")
59              try:
60                  sport_profile = sport_registry.get(sport_name)
61              except KeyError as e:
62                  logger.error(str(e))
63                  return []
64
65              idx_cfg = self.config.get("indexing", {})
66              provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
67              model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
68
69              api_key_env_var = f"{provider_name.upper()}_API_KEY"
70              api_key = os.environ.get(api_key_env_var)
71              if not api_key:
72                  logger.error(f"{api_key_env_var} not set; cannot run the {provider_name}
handoff.")
73                  return []
74              try:
75                  provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
76              except KeyError as e:
77                  logger.error(str(e))
78                  return []
79
80              video_duration = get_video_duration(video_path)
81              if video_duration <= 0:
82                  logger.error("Invalid video duration.")
83                  return []
84              fps, frame_width = self._probe_fps_width(video_path)
85
86              wasb_cfg = WasbConfig.from_indexing_cfg(idx_cfg)
87              runner: DetectorRunner = WasbRunner(wasb_cfg)
88
89              w = idx_cfg.get("wasb", {})
90              velocity_thresh = w.get("velocity_threshold", 0.02)
91              merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
92              min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
93              handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
94              ai_max_workers = idx_cfg.get("ai_max_workers", 5)
95              log_candidates = idx_cfg.get("log_yolo_candidates", True)
96              store_candidates = idx_cfg.get("store_yolo_candidates", True)
97
98              detection_params = {
```

```
                "detector": "wasb_hybrid",
                "velocity_thresh": velocity_thresh,
                "merge_gap": merge_gap,
                "min_action_window_duration": min_window_duration,
                "weights_path": wasb_cfg.weights_path,
            }

            ok, msg = runner.healthcheck()
            if not ok:
                logger.error("WASB WSL environment not ready: %s", msg)
                return []
            logger.info("WASB WSL env OK: %s", msg)

            # --- Phase 2: shuttle trajectory -> candidate rally windows ---
            t_start = time.time()
            out_dir = os.path.join("output", "wasb")
            csv_path = runner.run_predict(video_path, out_dir)
            if not csv_path:
                logger.error("WASB produced no trajectory; aborting.")
                return []

            points = runner.parse_trajectory_csv(csv_path)
            logger.info("Parsed %d trajectory points (%d visible).",
                        len(points), sum(1 for p in points if p.visible))

            all_candidate_windows = runner.trajectory_to_action_windows(
                points, fps=fps, frame_width=frame_width, chunk_start=0.0,
                velocity_thresh=velocity_thresh, merge_gap=merge_gap,
                min_window_duration=min_window_duration, chunk_index=0,
            )
            logger.info("Phase 2 (WASB) completed in %.1fs. Candidate windows: %d",
                        time.time() - t_start, len(all_candidate_windows))

            if log_candidates:
                for cw in all_candidate_windows:
                    logger.info(f"[WASB rally] {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
                                f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
                                f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")

            candidate_ids = [None] * len(all_candidate_windows)
            if store_candidates:
                for i, cw in enumerate(all_candidate_windows):
                    stats = {
                        "peak_velocity": cw["peak_velocity"], "mean_velocity":
cw["mean_velocity"],
                        "active_frame_count": cw["active_frame_count"], "track_id_count":
cw["track_id_count"],
                    }
                    candidate_ids[i] = self.db.add_candidate_segment(
                        video_id, detector="wasb_hybrid", start_time=cw["start"],
end_time=cw["end"],
                        chunk_index=cw["chunk_index"], confidence=min(1.0,
cw["peak_velocity"]),
                        stats=stats, detection_params=detection_params,
                    )

            if not all_candidate_windows:
                return []

            # --- Phase 3: AI provider handoff (shared pattern) ---
            handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
            os.makedirs(handoff_dir, exist_ok=True)
            logger.info("Phase 3: %s validation on %d candidate windows...",
```

```
                        provider_name, len(all_candidate_windows))

            def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
                w_start = max(0, window["start"] - handoff_padding)
                w_end = min(video_duration, window["end"] + handoff_padding)
                w_dur = w_end - w_start
                w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
                if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
                    return []
                prompt = sport_profile.get_prompt(chunk_duration=w_dur)
                segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
                                                     label=f"Handoff {wi+1}")
                valid = []
                for seg in segments:
                    try:
                        seg["start_time"] = float(seg["start_time"]) + w_start
                        seg["end_time"] = float(seg["end_time"]) + w_start
                        seg["_candidate_id"] = candidate_ids[wi]
                        valid.append(seg)
                    except (ValueError, TypeError, KeyError) as e:
                        logger.warning("Dropping segment with unparseable timestamps: %s ?
%s", seg, e)
                try:
                    os.remove(w_path)
                except OSError:
                    pass
                return valid

            ai_segments: List[dict] = []
            with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
                futures = [executor.submit(process_candidate_window, wi, cw)
                           for wi, cw in enumerate(all_candidate_windows)]
                for future in futures:
                    ai_segments.extend(future.result())
            try:
                os.rmdir(handoff_dir)
            except OSError:
                pass
            logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))

            # --- Phase 4: DB population (shared pattern) ---
            segments_data = []
            for segment in ai_segments:
                try:
                    start = float(segment["start_time"])
                    end = float(segment["end_time"])
                    dur = end - start
                    shot_count = segment.get("shuttle_exchange_count")
                    try:
                        shot_count = int(shot_count)
                    except (ValueError, TypeError):
                        shot_count = max(3, int(dur / 0.8))
                    metadata = {"shot_count": shot_count,
                                "shot_count_confidence":
segment.get("shot_count_confidence", "Unknown"),
                                "detector": f"wasb-hybrid-{model_name}"}
                    segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
                    if segment_id:
                        candidate_id = segment.get("_candidate_id")
                        if candidate_id is not None:
                            self.db.link_candidate_to_segment(candidate_id, segment_id)
```

```
217                         segments_data.append({"id": segment_id, "start_time": start,
"end_time": end,
218                                               "duration": dur, "metadata": metadata})
219                 except Exception as e:
220                     logger.warning("Skipping invalid segment struct: %s", e)
221
222             return segments_data
223
224
225     segmenter_registry.register(WasbHybridSegmenter)
226
```

6. user (2026-06-10T05:41:27.925Z)

```
__init__.py
__pycache__
base.py
gemini.py
motion.py
tracknet_hybrid.py
wasb_hybrid.py
yolo_hybrid.py
test_calibrate_wasb.py
test_export_wasb_segments.py
test_tracknet_hybrid.py
test_tracknet_runner.py
test_wasb_cache.py
test_yolo_hybrid.py
```

7. user (2026-06-10T05:41:42.451Z)

```
7:import json
8:import os
9:import os.path as osp
11:import numpy as np
13:from backend.pipeline.detectors import wasb_infer as wi
30:def test_det_plain_and_back_roundtrip():
45:def test_replay_accumulates_detections_across_windows(tmp_path):
62:def test_resume_index_equals_window_count(tmp_path):
70:def test_empty_cache(tmp_path):
80:def test_truncated_trailing_line_recovered_and_cleaned(tmp_path):
99:def test_noncontiguous_window_index_stops_replay(tmp_path):
112:def test_manifest_roundtrip_and_compat(tmp_path):
139:def test_read_manifest_missing_returns_none(tmp_path):
143:def test_reset_cache_removes_files(tmp_path):
156:def test_default_cache_dir_is_next_to_output():
163:def test_atomic_write_trajectory(tmp_path):
```

8. user (2026-06-10T05:41:43.387Z)

```
2:import csv
3:import pytest
5:from backend.eval.calibrate_wasb import (
34:def test_load_golden_gts_skips_incomplete(tmp_path):
39:def test_predict_fn_finds_moving_segments(tmp_path):
46:def test_run_end_to_end_synthetic(tmp_path):
```

9. user (2026-06-10T05:41:44.797Z)

```
2:import csv
4:from backend.eval import export_wasb_segments as ex
5:from backend.tools.rally_slicer import load_rally_labels
31:def test_seconds_to_mmss():
37:def test_segments_to_rows_numbers_in_time_order():
```

```
45:def test_exported_csv_feeds_straight_into_the_slicer(tmp_path):
52:def test_build_comparison_classifies_match_miss_and_false_positive():
68:def test_run_writes_segments_and_comparison(tmp_path):
86:def test_run_min_crossings_gates_low_crossing_segments(tmp_path):
```

10. user (2026-06-10T05:41:45.817Z)

```
1:"""Tests for the TrackNetHybridSegmenter pipeline. The WSL/TensorFlow runner is
9:from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
28:    fn = patch("backend.pipeline.segmenters.tracknet_hybrid.TrackNetRunner")(fn)
29:    fn = patch.object(TrackNetHybridSegmenter, "_probe_fps_width", return_value=(30.0,
1280.0))(fn)
30:    fn = patch("backend.pipeline.segmenters.tracknet_hybrid.get_video_duration")(fn)
31:    fn = patch("backend.pipeline.segmenters.tracknet_hybrid.extract_video_chunk")(fn)
32:    fn = patch("backend.pipeline.segmenters.tracknet_hybrid.provider_registry")(fn)
47:def test_pipeline_happy_path(mock_runner_cls, mock_probe, mock_dur, mock_extract,
63:    seg = TrackNetHybridSegmenter(db, {"indexing": {}})
70:    assert kwargs["detector"] == "tracknet_hybrid"
76:def test_pipeline_no_api_key_returns_empty(mock_runner_cls, mock_probe, mock_dur,
mock_extract,
82:    seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
87:def test_pipeline_healthcheck_failure_aborts(mock_runner_cls, mock_probe, mock_dur,
mock_extract,
95:    seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
103:def test_pipeline_no_trajectory_aborts(mock_runner_cls, mock_probe, mock_dur, mock_extract,
110:    seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
115:def test_pipeline_store_candidates_disabled(mock_runner_cls, mock_probe, mock_dur,
mock_extract,
129:    seg = TrackNetHybridSegmenter(db, {"indexing": {"store_yolo_candidates": False}})
137:def test_pipeline_drops_unparseable_segments(mock_runner_cls, mock_probe, mock_dur,
mock_extract,
153:    seg = TrackNetHybridSegmenter(db, {"indexing": {}})
158:def test_probe_fps_width_fallbacks():
159:    with patch("backend.pipeline.segmenters.tracknet_hybrid.cv2.VideoCapture") as mock_cap:
164:        fps, width = TrackNetHybridSegmenter._probe_fps_width("missing.mp4")
168:def test_segmenter_identity():
169:    seg = TrackNetHybridSegmenter(MagicMock(), {})
170:    assert seg.name == "tracknet_hybrid"
```

11. user (2026-06-10T05:41:57.764Z)

```
Found 1 file
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_tracknet_runner.py
```

12. user (2026-06-10T05:41:58.786Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_yolo_hybrid.py:7

Found 7 total occurrences across 1 file.
```

13. user (2026-06-10T05:41:59.523Z)

```
215                 pass
216             logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
217
218             # --- Phase 4: DB population (identical pattern to HybridYoloSegmenter) ---
219             segments_data = []
220             for segment in ai_segments:
221                 try:
222                     start = float(segment["start_time"])
223                     end = float(segment["end_time"])
224                     dur = end - start
225                     shot_count = segment.get("shuttle_exchange_count")
```

```
226                     if shot_count is None:
227                         shot_count = max(3, int(dur / 0.8))
228                     else:
229                         try:
230                             shot_count = int(shot_count)
231                         except (ValueError, TypeError):
232                             shot_count = max(3, int(dur / 0.8))
233                     metadata = {
234                         "shot_count": shot_count,
235                         "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
236                         "detector": f"tracknet-hybrid-{model_name}",
237                     }
238                     segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
239                     if segment_id:
240                         candidate_id = segment.get("_candidate_id")
241                         if candidate_id is not None:
242                             self.db.link_candidate_to_segment(candidate_id, segment_id)
243                         segments_data.append({
244                             "id": segment_id, "start_time": start, "end_time": end,
245                             "duration": dur, "metadata": metadata,
246                         })
247                 except Exception as e:
248                     logger.warning("Skipping invalid segment struct: %s", e)
249
```

14. user (2026-06-10T05:42:01.087Z)

```
38:Hybrid segmenter internal 4-phase (yolo_hybrid / tracknet_hybrid / wasb_hybrid ? same shape):
40:P1 chunk video (yolo only; tracknet/wasb = whole-video single pass, chunk_index=0)
48:### 1B. WASB shuttle substrate (P2 of wasb_hybrid)
50:WasbHybridSegmenter.process_video               @backend/pipeline/segmenters/wasb_hybrid.py
51: -> WasbRunner.healthcheck()  (gate; not ok -> return [])
@backend/pipeline/detectors/wasb_runner.py
52: -> WasbRunner.run_predict(video, out_dir)
53:      stage video + wasb_infer.py into WSL fs -> `wsl bash -c 'source conda.sh && python
wasb_infer.py ...'`
54:      -> @backend/pipeline/detectors/wasb_infer.py::run (WSL): sliding windows -> GPU
detector pass (CHECKPOINTED det_raw.jsonl) -> CPU tracker (always full re-run) -> trajectory CSV
55: -> TrackNetRunner.parse_trajectory_csv(csv)
@backend/pipeline/detectors/tracknet_runner.py  (shared, re-exported by WasbRunner)
58:(`tracknet_hybrid` identical, swapping WasbRunner->TrackNetRunner; `_probe_fps_width`
fallback (30.0, 1280.0).)
62:GT CSV -> annotations.load_annotations / calibrate_wasb.load_golden_gts -> List[Interval]
68:Calibration: calibrate_wasb.run -> calibration.{select_best, improvement_report,
cross_validate}  @backend/eval/calibration.py
74:Entrypoints: `@run_eval.py` (single video score), `@run_phase3_eval.py` (manifest batch, can
segment), `@run_regression.py` (baseline gate, exit 1=regression),
`@backend/eval/calibrate_wasb.py` + `@backend/eval/export_wasb_segments.py` (WASB tuning
drivers).
81:- `@backend/main.py` ? Thin CLI + FastAPI bootstrap (orchestrator; modes via FLAGS). Static
UI mounts `/static` from `backend/api/static`. Logging is initialized with `basicConfig` (INFO,
timestamped format); backend modules use `logger = logging.getLogger(__name__)` (replaces stray
prints in hot paths like motion, stitcher, wasb_infer, config loader). User-facing CLI summaries
may still use `print` for direct output.
109:- `@backend/pipeline/segmenters/__init__.py` ? imports all 5
(`motion,gemini,yolo_hybrid,tracknet_hybrid,wasb_hybrid`) = the de-facto registration manifest;
re-exports them + `segmenter_registry` via `__all__`.
110:[Omitted long matching line]
111:- `@backend/pipeline/segmenters/tracknet_hybrid.py::TrackNetHybridSegmenter` (name
`'tracknet_hybrid'`) ? copy-paste twin of wasb_hybrid; imports `TrackNetRunner`/`TrackNetConfig`
from `pipeline/detectors/tracknet_runner`; `indexing.tracknet.velocity_threshold(0.02)`;
`detection_params` adds `queue_length`. ? any P3/P4 fix must be applied to BOTH (and arguably
yolo_hybrid).
```

121:[Omitted long matching line]
122:[Omitted long matching line]
123:[Omitted long matching line]
124:[Omitted long matching line]
130:- `@backend/eval/calibrate_wasb.py` ? WASB tuning CLI. `::BASELINE=(0.02,2.0,2.0)`,
`::default_grid` (45 cfgs), `::make_predict_fn` (parses trajectory once, closes over
`trajectory_to_action_windows`), `::load_golden_gts` (§`start_time,end_time` cols; ? SILENTLY
skips bad rows ? opposite of annotations.load_annotations). ? `{load_golden_gts,
make_predict_fn, BASELINE}` imported by 4 downstream drivers (calibrate_local,
export_wasb_segments, gemini_refine, audio/e1_probe).
132:- `@backend/eval/export_wasb_segments.py` ? tuned cfg -> golden/slicer CSV + comparison +
optional clip cut. `::TUNED=(0.05,1.0,1.0)` (? from ONE video), `::SEG_FIELDS`,`::COMP_FIELDS`,
`::build_comparison`, `::seconds_to_mmss`, `::_maybe_slice` (lazy-imports rally_slicer).
`--slice` requires `--video`. ? `write_segments_csv` output loads straight into
`rally_slicer.load_rally_labels` AND re-imported by gemini_refine ? schema is load-bearing.
137:[Omitted long matching line]
138:[Omitted long matching line]
139:[Omitted long matching line]
141:[Omitted long matching line]
145:[Omitted long matching line]
146:- `@backend/eval/audio/e1_probe.py` ? E1 GO/NO-GO diagnostic (no training/fusion).
`::cluster_hits` (groups thwacks; ? `min_hits>=2` so a lone service-feed hit isn't a rally),
`::run` (discrimination ratio + recall recovery + audio-only P/R/F1), `::main`. reuses
`hit_detector` + `calibrate_wasb.{load_golden_gts,make_predict_fn,BASELINE}` + `metrics`.
177:[Omitted long matching line]
184:§ **Trajectory CSV** (produced by tracknet/wasb predict, consumed by
`parse_trajectory_csv`):
193:`detection_params` keys by detector:
wasb=`{detector,velocity_thresh,merge_gap,min_action_window_duration,weights_path}`; tracknet
adds `queue_length`; yolo=`{velocity_thresh,merge_gap,frame_skip,tracker,detector_model,detector
_score_threshold,min_action_window_duration}`.
202:- golden/slicer (`calibrate_wasb.load_golden_gts`, `rally_slicer.load_rally_labels`,
`export.SEG_FIELDS`): `rally_number,start_time,end_time[,duration,start_mmss,end_mmss]`;
SILENTLY skips bad rows.
218:## 4. WASB WSL CONTRACT (external dep `~/models/WASB-SBDT`, MIT; NOT in this repo ? claims
below are unverified against external source, only the repo-side caller wasb_infer.py was
checked)
219:- Env: WSL conda `wasb`, cwd `~/models/WASB-SBDT/src`. Hydra `compose(config_name='eval',
overrides=[dataset=<sport>, model=wasb, detector.model_path=<weights>, runner.device=cuda,
runner.gpus=[0]])`. `gpus=[0]` because box is single-GPU. (overrides VERIFIED in
@backend/pipeline/detectors/wasb_infer.py::_build_cfg.)
220:- `model=wasb` -> `name=hrnet`, `frames_in=3`, `frames_out=3`, input 512x288 (WxH), ImageNet
normalize; affine resize maps heatmap coords -> original-frame coords. (model dims read from
cfg["model"] at runtime in wasb_infer.run.)
223:- `dataloaders/dataset_loader.py::ImageDataset(cfg, samples, input_wh, output_wh, transform,
seq_transform, is_train)`; `samples=[{frames:[paths], annos:[{frame_path,
center:Center(is_visible,x,y)}]}]`. (caller constructs exactly this ? VERIFIED in
wasb_infer.run.)
224:- Weights `pretrained_weights/wasb_badminton_best.pth.tar` ? NOT committed, academic-data-
trained -> ? confirm terms before COMMERCIAL deploy. monotrack weights = noncommercial, avoided.
231:- Collector also emits golden rally labels (`rally_number,start_time,end_time,...`) ->
consumed by `@backend/eval/calibrate_wasb.py` / `export_wasb_segments.py` /
`@backend/tools/rally_slicer.py`.
244:- **fps / frame_width must be PROBED not assumed:** `_probe_fps_width` / runner fallbacks
(30.0, 1280.0) silently mis-scale velocity thresholds; calibrate_wasb CLI bakes `59.94/1920`.
The report's `fps_fallback_used` flag surfaces this.

[Showing results with pagination = limit: 40]

15. user (2026-06-10T05:42:20.724Z)

59:`calibrate_wasb.py` do honest leave-one-video-out (LOVO), but nothing *enforces*
123:- **Unify hybrid segmenters (#7 in LOW_REGRET):** owner-deferred.

16. user (2026-06-10T05:42:22.090Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md:177:[Omitted long matching
line]
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md:328:| (no dedicated test) |
individual `pipeline/validators/*` impls, `pipeline/stitcher.compiler`, `main.py`,
`run_process_and_stitch.py`, `pipeline/detectors/wasb_runner`,
`pipeline/segmenters/{wasb_hybrid,motion}`, `eval/audio/e1_probe` (diagnostic) ? ? exercised
only indirectly |
```

17. user (2026-06-10T05:42:23.608Z)

```
44-        video_id="md5abc",
45-        video_path=None,
46:        config={"ai_provider": "gemini", "ai_model": "gemini-flash", "indexing":
{"default_segmenter": "wasb_hybrid"}},
47-        val_results=[_vr("format_check", True), _vr("relevance_check", True)],
48-        val_context={"ffprobe_meta": _FFPROBE_OK},
49-        play_segments=[{"duration": 5.0, "metadata": {"detector": "wasb-hybrid-gemini"}}],
50-        candidates=[{"id": 1}, {"id": 2}],
51:        segmenter_requested="wasb_hybrid",
52:        segmenter_actual="wasb_hybrid",
53-        was_reingest=False,
54-        segmentation_ran=True,
--
105-def test_segmenter_divergence_fires_only_without_explicit_override(monkeypatch):
106-    monkeypatch.setenv("GEMINI_API_KEY", "x")
107:    cfg = {"ai_provider": "gemini", "indexing": {"default_segmenter": "wasb_hybrid"}}
108-    # No explicit request, but a different segmenter ran -> a fallback diverged: flag it.
109-    env = _record(segmenter_requested=None, segmenter_actual="gemini", config=cfg)
--
148-    paths = emit_indexer_report(
149-        db=db, video_id="md5abc", video_path=str(video),
150:        config={"ai_provider": "gemini", "indexing": {"default_segmenter": "wasb_hybrid"}},
151-        val_results=[_vr("format_check", True)], val_context={"ffprobe_meta": _FFPROBE_OK},
152:        segmenter_requested="wasb_hybrid", segmenter_actual="wasb_hybrid",
153-        was_reingest=False, segmentation_ran=True,
154-    )
--
172-        db=db, video_id="md5cfg", video_path=str(video), config=cfg,
173-        val_results=[_vr("format_check", True)], val_context={"ffprobe_meta": _FFPROBE_OK},
174:        segmenter_requested="wasb_hybrid", segmenter_actual="wasb_hybrid",
175-        was_reingest=False, segmentation_ran=True,
176-    )
```

18. user (2026-06-10T05:42:24.907Z)

```
25:def test_to_wsl_mnt_path_windows():
30:def test_to_wsl_mnt_path_passthrough():
36:def test_config_from_indexing_cfg():
48:def test_config_defaults_when_absent():
55:def test_parse_trajectory_csv(tmp_path):
64:    pts = TrackNetRunner.parse_trajectory_csv(str(csv_file))
71:def test_trajectory_to_action_windows_detects_moving_shuttle():
78:    windows = TrackNetRunner.trajectory_to_action_windows(
90:def test_trajectory_to_action_windows_ignores_stationary_shuttle():
93:    windows = TrackNetRunner.trajectory_to_action_windows(
100:def test_trajectory_to_action_windows_splits_on_gap():
106:    windows = TrackNetRunner.trajectory_to_action_windows(
120:def test_healthcheck_ok():
128:def test_healthcheck_failure_returns_message():
136:def test_healthcheck_no_wsl_binary():
144:def test_run_predict_requires_weights():
149:def test_run_predict_rejects_bad_extension(tmp_path):
154:def test_run_predict_success(tmp_path):
165:def test_run_predict_failure_returns_none(tmp_path):
171:def test_run_predict_missing_output_csv(tmp_path):
```

```
178:def test_stage_video_success():
185:def test_stage_video_failure():
191:def test_run_predict_stages_when_enabled(tmp_path):
206:def test_tracknet_runner_implements_detector_runner_abc():
218:def test_wasb_runner_implements_detector_runner_abc():
```

19. user (2026-06-10T05:43:14.312Z)

```
Structured output provided successfully
```

20. assistant (2026-06-10T05:43:18.033Z)

```
Verification complete.
```